

VAGA · DESENVOLVEDOR(A) FULL-STACK — TRAINEE

Reserva de Salas

Desafio técnico full-stack

Um desafio pequeno de propósito. Não queremos ver quantas features você entrega — queremos ver como você pensa, modela e resolve.

Prazo: 24h

Entrega: GitHub + vídeo

Stack: TypeScript + React + Postgres

01 — Contexto

O que você vai construir

Uma aplicação web para **gerenciar reservas de salas** — daquelas salas de reunião que vivem em conflito de agenda. O usuário cadastra salas e reserva horários nelas. Simples na superfície; o valor está nos detalhes.

Leia o case inteiro antes de começar. Boa parte da avaliação está em **como você interpreta o problema** e nas decisões que toma diante do que **não** está especificado aqui.

02 — Stack

O que é obrigatório e o que é livre

Obrigatório

- ◆ **TypeScript** em todo o projeto
- ◆ **React** — com `Vite` ou `Next.js`
- ◆ **PostgreSQL** — via `Supabase` ou `Neon`

Livre / a seu critério

- ◆ Acessar o banco direto pelo BaaS **ou** ter uma camada de API própria (Fastify, route handlers do Next, Edge Functions...)
- ◆ Biblioteca de UI, estilização, gerência de estado

Não exigimos API própria — mas construir uma, com a regra de negócio validada no servidor, conta como diferencial (ver critério de arquitetura).

03 — Escopo

O que precisa funcionar

OBRIGATÓRIO — O PISO

- ♦ **CRUD de reservas** persistido no banco — criar, listar, editar e remover
- ♦ **Listagem com filtro** por sala e ordenação por horário
- ♦ **Estados derivados** da reserva: em andamento agora, próximas, encerradas
- ♦ **Validação de entrada** consistente (campos obrigatórios, `fim > início`)

DIFERENCIAL — PRA QUEM QUER IR ALÉM

- ♦ CRUD de salas (nome, capacidade)
- ♦ **Camada de API própria** com a checagem de conflito rodando **no servidor**
- ♦ Deploy acessível (Vercel ou similar)
- ♦ Autenticação, testes da regra de conflito, visualização em timeline/calendário

Diferencial é diferencial: não tente fazer tudo. Preferimos o obrigatório bem feito a tudo pela metade.

04 — Regras de negócio

O coração do desafio

Regra 1 — Conflito de horário

Duas reservas **na mesma sala** não podem se sobrepor no tempo. Ao criar ou editar, o sistema deve **impedir** a reserva conflitante e avisar o usuário com clareza.

Regra 2 — Capacidade

Cada reserva informa um número de participantes. Uma reserva não deveria estourar a capacidade da sala. Como você lida com isso — bloqueio rígido, aviso, ou outra abordagem — é **decisão sua**.

♦ DECISÕES QUE VOCÊ PRECISA TOMAR (E DOCUMENTAR)

De propósito, não respondemos tudo. Queremos ver seu critério. No `README`, registre as premissas que assumiu para perguntas como:

- ♦ Reservas que **encostam** (uma termina 14h, outra começa 14h) são conflito ou não?
- ♦ Existe **horário de funcionamento**? Pode reservar de madrugada? Fim de semana?
- ♦ O que acontece ao **editar** uma reserva para um horário que agora conflita?
- ♦ Como o front comunica um conflito de forma que o usuário entenda o que fazer?

Não há resposta única certa. Há decisões bem ou mal justificadas.

Pergunta de raciocínio (não precisa codar)

No README ou no vídeo, responda em poucas linhas: **como você evoluiria este sistema para suportar reservas recorrentes** (ex.: "toda terça às 14h, pelos próximos 3 meses")? O que mudaria no modelo de dados e na checagem de conflito?

05 — Entrega

O que enviar

- 1** OBRIGATÓRIO
Repositório no GitHub — código completo, com histórico de commits que conte a história do desenvolvimento.
 - 2** OBRIGATÓRIO
README bem estruturado — visão geral do projeto, stack escolhida, passo a passo de setup que rode do zero, **suas premissas e decisões** e a resposta da pergunta de raciocínio.
 - 3** OBRIGATÓRIO
Vídeo curto (até 5 min) — demonstrando o sistema funcionando: criar uma reserva, o filtro por sala e um caso de conflito sendo bloqueado.
- ★ OPCIONAL
Link de deploy — sistema acessível online (Vercel ou outra hospedagem).

06 — Avaliação

Como vamos pontuar

Transparência total — é assim que avaliamos. Mire em decisões maduras, não em volume.

CRITÉRIO	O QUE OLHAMOS	PESO
Regras de negócio	Conflito correto, capacidade, edge cases bem tratados	25%
Arquitetura & código	Camadas, onde mora a validação, tipagem real, organização · API no servidor = diferencial	25%
Banco de dados	Schema coerente, relação sala→reserva, constraints, migrations	20%
Front-end & UX	Estados de loading / vazio / erro, feedback ao usuário, responsividade	20%
Interpretação & comunicação	Premissas documentadas, qualidade do README e do vídeo	10%

07 — Uso de IA

Pode usar IA?

Pode, à vontade. Copilot, Claude, o que preferir — é assim que se trabalha hoje. O que avaliamos é o resultado: um sistema que funciona, código organizado e, principalmente, um **README bem estruturado** que deixe claras as suas decisões e o seu raciocínio.



Boa sorte!

Dúvidas sobre o enunciado, fale com a gente. Dúvidas sobre **decisões de produto** são suas para resolver — só documente.